

C Programming Notes

Theory Notes

Unit I – Basics of C Programming

Unit II – Control Structures

Unit III – Arrays and Structure

Unit IV – Functions

Unit V – Pointers

Structured notes covering the prescribed C programming topics.

Unit I - Basics of C Programming

1.1 Fundamentals of Algorithms

Algorithm: An algorithm is a finite sequence of clear and ordered steps used to solve a problem. Each step must be precise, and the process should eventually produce a result.

- Input: Data supplied to the problem.
- Processing: Operations performed on the input.
- Output: Result produced after processing.
- Finiteness: An algorithm must terminate after a finite number of steps.
- Definiteness: Every instruction should have an unambiguous meaning.

Example - finding the larger of two numbers: Start → read two numbers → compare them → display the larger number → stop.

1.2 Notations for Algorithmic Statements

Algorithms can be represented using simple textual steps or a flowchart. A flowchart uses standard symbols to show the sequence of operations.

- Terminal: Represents Start or Stop.
- Input/Output: Represents reading data or displaying results.
- Process: Represents a calculation or assignment.
- Decision: Represents a condition with alternative paths.
- Flow line: Shows the direction of execution.

1.3 Introduction to C

C is a general-purpose, procedural programming language. It is widely used for system software, embedded programming, utilities and applications where efficient control over memory and hardware is important.

General structure of a C program: A typical program contains preprocessor directives, a main function, declarations, executable statements and a return statement.

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C");
    return 0;
}
```

1.4 Header Files and main() Function

A header file contains declarations and definitions that can be reused by a program. The `stdio.h` header provides standard input/output functions such as `printf()` and `scanf()`.

The `main()` function is the starting point of execution in a C program. The statements inside `main()` are executed when the program runs.

1.5 Character Set, Tokens and Keywords

The C character set includes letters, digits, special symbols and whitespace characters used to form source code.

Tokens are the smallest meaningful units of a C program. Major token categories are keywords, identifiers, constants, string literals, operators and special symbols.

Keywords are reserved words with predefined meanings, such as `int`, `char`, `if`, `else`, `while`, `for`, `return` and `struct`.

Identifiers are programmer-defined names for variables, functions, arrays and other entities. They may contain letters, digits and underscores, but cannot begin with a digit.

1.6 Constants, Variables and Data Types

A constant is a value that does not change during a particular program operation. A variable is a named memory location whose value can change during execution.

Basic data types: `int` for integers, `char` for characters, `float` for single-precision real values and `double` for double-precision real values.

A declaration specifies the data type and name of an object. Initialization gives it an initial value.

```
int count = 10;
char grade = 'A';
float price = 25.5f;
double distance = 123.456;
```

1.7 Declaring Variables and Data Type Conversion

Variables should be declared before they are used. C supports conversion between compatible data types.

Implicit conversion is performed automatically by the compiler when required. Explicit conversion uses a cast operator to request a particular type.

```
int a = 5, b = 2;
double result = (double)a / b;
```

1.8 Basic Input and Output Functions

The `printf()` function displays formatted output. The `scanf()` function reads formatted input from standard input. Format specifiers tell these functions the type of data being processed.

```
int n;
printf("Enter a number: ");
scanf("%d", &n);
printf("Number = %d", n);
```

Common format specifiers include `%d` for `int`, `%c` for `char`, `%f` for floating-point values and `%lf` for double input with `scanf()`.

1.9 Assignment and Operators

The assignment operator `=` stores a value in a variable. C provides arithmetic, relational, logical, bitwise, shift and other operators.

- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Relational: `<`, `>`, `<=`, `>=`, `==`, `!=`
- Logical: `&&`, `||`, `!`
- Bitwise: `&`, `|`, `^`, `~`
- Shift: `<<`, `>>`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, `%=`

The modulo operator (%) gives the remainder of integer division. Shift operators move the binary representation of a value to the left or right.

Unit II - Control Structures

2.1 Conditional Statements

Conditional statements allow a program to choose different actions according to a condition. The condition is generally evaluated as true or false.

if statement: Executes a block only when its condition is true.

```
if (marks >= 40)
{
printf("Pass");
}
```

if-else statement: Selects one of two blocks depending on the condition.

```
if (n % 2 == 0)
printf("Even");
else
printf("Odd");
```

else-if ladder: Tests multiple conditions in sequence. The first true condition selects the corresponding block.

Nested if: An if statement can be placed inside another if or else block.

2.2 switch Statement

The switch statement provides multi-way selection. An expression is compared with case labels. The break statement normally ends the selected case.

```
switch (choice)
{
case 1:
printf("One");
break;
case 2:
printf("Two");
break;
default:
printf("Invalid choice");
}
```

2.3 Looping Statements

Loops repeatedly execute a block while a specified condition or loop rule is satisfied.

while loop: The condition is checked before the loop body, so the body may execute zero times.

```
while (condition)
{
/* statements */
}
```

do-while loop: The body executes first and the condition is checked afterward. Therefore, the body executes at least once.

```
do
{
/* statements */
} while (condition);
```

for loop: Combines initialization, condition and update in one loop statement and is useful when the loop control is clearly defined.

```
for (int i = 1; i <= 5; i++)  
{  
    printf("%d ", i);  
}
```

2.4 Branching Statements

goto transfers control to a labeled statement within the same function. It should be used carefully because excessive use can make program flow difficult to understand.

break terminates the nearest loop or switch statement.

continue skips the remaining statements of the current loop iteration and starts the next iteration according to the loop mechanism.

Unit III - Arrays and Structure

3.1 Characteristics of an Array

An array is a collection of elements of the same data type stored under one name. Elements are accessed using an index. In C, array indexing starts from 0.

- Elements of an array have the same data type.
- Array elements are stored in contiguous memory locations.
- The size of a fixed array is specified when it is declared.
- The last valid index of an array of size n is $n-1$.

3.2 One-Dimensional Arrays

A one-dimensional array represents a linear sequence of elements.

```
int marks[5];  
int values[5] = {10, 20, 30, 40, 50};  
  
printf("%d", values[2]);
```

The declaration reserves space for five integers, while the initialized array assigns values to its elements.

3.3 Two-Dimensional and Multi-Dimensional Arrays

A two-dimensional array is commonly used to represent a table or matrix. It is declared using two dimensions: rows and columns.

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

C also supports arrays with more than two dimensions. Such arrays are useful when data naturally has multiple dimensions.

3.4 Array Declaration, Initialization and Operations

Arrays can be declared without initial values or initialized during declaration. Common operations include traversal, searching, updating, copying and sorting.

When traversing an array, a loop is commonly used to access elements one by one. Care must be taken not to access an index outside the valid range.

3.5 Character Arrays and Strings

A string in C is represented by a character array terminated by the null character '\0'. String functions are provided by the string.h header.

```
char name[20] = "C Programming";  
  
printf("%s", name);
```

Common string functions include `strlen()` for length, `strcpy()` for copying, `strcat()` for concatenation and `strcmp()` for comparison. These functions should be used with arrays large enough to hold the resulting string.

3.6 Structure

A structure is a user-defined data type that groups related variables of different data types under one name. Each member has its own type and memory is allocated for the structure as a whole.

```
struct Student
{
    int roll;
    char name[30];
    float marks;
};

struct Student s1;
```

Structure members are accessed using the dot (.) operator. A pointer to a structure uses the arrow (->) operator to access members.

3.7 Declaration and Initialization of Structures

```
struct Student s1 = {1, "Amit", 85.5f};
```

Structures can also be declared as arrays when multiple records of the same structure type are required.

```
struct Student students[3];
```

3.8 typedef and enum

typedef creates an alternative name for an existing data type, which can make declarations shorter and clearer.

```
typedef unsigned int uint;
uint count = 10;
```

enum defines a set of named integer constants. It is useful when a variable can take one value from a limited set of meaningful choices.

```
enum Day {MON, TUE, WED, THU, FRI};
```


Unit IV - Functions

4.1 Concept and Need of Functions

A function is a named block of statements designed to perform a specific task. Functions divide a large program into smaller, manageable units.

- Improve modularity and program organization.
- Reduce repetition by allowing code reuse.
- Make testing and debugging easier.
- Improve readability and maintenance.

4.2 Library Functions

Library functions are predefined functions supplied through standard C libraries. The required header file is included before using the relevant functions.

- Math functions: `sqrt()`, `pow()`, `ceil()`, `floor()` and related functions from `math.h`.
- String functions: `strlen()`, `strcpy()`, `strcat()`, `strcmp()` from `string.h`.
- Character functions: functions such as `getchar()` and related character operations.
- Memory functions: functions such as `malloc()`, `calloc()`, `realloc()` and `free()` are provided through `stdlib.h`.

4.3 User-Defined Functions

A user-defined function is written by the programmer for a particular task. A function generally has a declaration (prototype), a definition and one or more calls.

```
int add(int a, int b); /* declaration */

int add(int a, int b) /* definition */
{
    return a + b;
}

int result = add(10, 20); /* call */
```

A function may return a value or may have a void return type. Its parameters specify the data supplied to the function.

4.4 Scope of Variables

Local variable: Declared inside a function or block and normally accessible only within that scope.

Global variable: Declared outside functions and can be accessed by functions according to its visibility and linkage.

Keeping variables local where possible improves program organization and reduces unintended interactions between functions.

4.5 Function Parameters and Calling Methods

In C, function arguments are passed by value. The called function receives copies of the argument values. Changes made to those parameter variables do not directly change the caller's original variables.

```
void change(int x)
{
    x = 100;
```

```
}  
  
int a = 10;  
change(a);  
/* a remains 10 */
```

A function can modify caller data when the address of that data is passed using a pointer. This is commonly described as achieving reference-like behavior through pointers.

4.6 Function Return Values and Types

A function's return type specifies the type of value it returns. The return statement transfers control back to the calling function and may provide a value.

```
double square(double x)  
{  
    return x * x;  
}
```

If a function does not return a value, its return type is void.

4.7 Recursive Functions

Recursion occurs when a function calls itself to solve a problem by reducing it to smaller instances of the same problem. A recursive function must have a base condition that stops further calls.

```
int factorial(int n)  
{  
    if (n <= 1)  
        return 1;  
    return n * factorial(n - 1);  
}
```

Each recursive call uses additional stack space, so recursion should be designed carefully.

Unit V - Pointers

5.1 Introduction to Pointers

A pointer is a variable that stores the memory address of another variable. Pointers are important for dynamic memory, arrays, strings, functions and efficient data manipulation.

The `&` operator obtains the address of an object, while the `*` operator is used to declare a pointer and to dereference it.

```
int n = 25;
int *p = &n;

printf("%d", *p);
```

Here, `p` stores the address of `n`, and `*p` accesses the value stored at that address.

5.2 Declaring, Initializing and Accessing Pointers

A pointer should be initialized with a valid address before dereferencing it. A pointer that does not currently point to a valid object can be initialized to `NULL`.

```
int value = 50;
int *ptr = &value;

if (ptr != NULL)
    printf("%d", *ptr);
```

5.3 Pointer Arithmetic

Pointer arithmetic is related to the size of the pointed-to data type. Adding 1 to an `int` pointer advances it to the next `int` element, not merely to the next byte.

- `p + 1` moves to the next element of the pointed-to type.
- `p - 1` moves to the previous element.
- Two pointers can be compared or subtracted when they refer to elements of the same array.

Pointer arithmetic is especially useful for traversing arrays.

5.4 Pointers and Arrays

The name of an array is closely related to the address of its first element in expressions. Array elements can therefore be accessed using pointer notation.

```
int a[3] = {10, 20, 30};
int *p = a;

printf("%d", *(p + 1));
```

The expression `*(p + 1)` accesses the second element of the array.

5.5 Pointers and Text Strings

Strings can be manipulated using character pointers because a string is stored as a sequence of characters terminated by `'\0'`.

```
char text[] = "Hello";
char *p = text;

while (*p != '\0')
{
    putchar(*p);
    p++;
}
```

```
}
```

5.6 Functions and Pointers

Pointers allow a function to receive the address of a variable. This lets the function modify the caller's data and is useful for returning multiple results through output parameters.

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

The caller passes addresses, and the function accesses the original values through dereferencing.

5.7 Pointers to Structures

A pointer can point to a structure object. Structure members can be accessed through the arrow operator (->).

```
struct Student
{
    int roll;
};

struct Student s = {10};
struct Student *p = &s;

printf("%d", p->roll);
```

The arrow operator is a convenient way to access a structure member through a structure pointer.

Quick Revision - Core Concepts

- C program: Source code is organized using functions, declarations, statements and library facilities.
- Operators: Perform arithmetic, comparison, logical, bitwise and assignment operations.
- Control structures: if, if-else, else-if, switch, while, do-while and for control program flow.
- Arrays: Store multiple elements of the same type using indexed access.
- Structures: Group related members of different types.
- Functions: Divide programs into reusable and manageable units.
- Recursion: A function solves a problem through smaller instances of itself.
- Pointers: Store addresses and support indirect access, arrays, functions and structures.

These notes follow the five C programming units shown in the supplied syllabus page and focus on theory and core concepts only.